

Pemrograman Web Lanjut

SIB245007 | D-IV Sistem Informasi Bisnis

Pertemuan 1: **Arsitektur Web Modern dan Ekosistem Laravel**

Rencana Pembelajaran Semester (RPS) & Materi Bab 1

Bagian 1

Identitas Mata Kuliah

Program Studi	D-IV Sistem Informasi Bisnis
Kode Mata Kuliah	SIB245007
Nama Mata Kuliah	Pemrograman Web Lanjut

Mata kuliah ini memakai **Simple POS**, aplikasi kasir/point-of-sale UMKM yang kodenya nyata dan berkembang commit demi commit, sebagai studi kasus berjalan sepanjang satu semester.

Rencana Pembelajaran (1/2)

Pertemuan	Materi
1	Arsitektur Web Modern
2	HTTP & Arsitektur MVC
3	Frontend & Templating
4	Desain Basis Data & Migrasi
5	ORM & Relasi Data
6	Validasi & Keamanan Input
7	Autentikasi & Otorisasi
8	UTS
9	Pengolahan & Ekspor Data

Rencana Pembelajaran (2/2)

Pertemuan	Materi
10	Arsitektur & Desain API
11	Perencanaan Proyek PBL
12	Pengembangan Fitur PBL
13	Integrasi & Pengujian PBL
14	Optimasi & Deployment PBL
15	Finalisasi Proyek PBL
16	Persiapan Ujian Akhir PBL
17	UAS

Pertemuan 1–10 membangun fondasi teknis Laravel; Pertemuan 11–17 mengalihkannya menjadi proyek *Project Based Learning* (PBL) mandiri.

Komponen Evaluasi

Basis Evaluasi	Bobot
Aktivitas Partisipatif (Case Method)	0%
Hasil Proyek (Project Based Learning)	55%
Kognitif – Tugas mingguan (increment)	15%
Kognitif – Quiz	0%
Kognitif – UTS (progress proyek + presentasi)	5%
Kognitif – UAS (proyek final + presentasi)	25%
Total	100%

Tugas mingguan: implementasi bertahap (*increment*) aplikasi Simple POS, dikumpulkan sebagai bukti kode dan dokumentasi sesuai rubrik.

Bagian 2

Restoran Keluarga vs. Food Court

Restoran keluarga

- Satu dapur, satu kasir, satu tim
- Semua orang tahu semua hal
- Ramai? Tambah kompor, bukan cabang baru

Food court

- Setiap tenant: dapur, kasir, resep sendiri
- Unit-unit independen
- Satu tenant sepi tidak mengganggu yang lain

Simple POS dipilih sebagai **monolith** (restoran keluarga) bukan karena keterbatasan, tapi karena skalanya cocok: satu warung dengan satu-dua kasir tidak butuh sepuluh layanan terpisah.

Yang Akan Kamu Pelajari

1. Membandingkan arsitektur **monolith**, **microservices**, dan **serverless** untuk menjelaskan alasan Laravel dipilih sebagai kerangka kerja Simple POS
2. Memahami alasan **Laravel 13** dengan **SQLite** sebagai basis data zero-setup dipilih untuk Simple POS
3. Mengenali struktur folder proyek Laravel (`routes/` , `app/Http/Controllers/` , `database/migrations/`) sebagai perwujudan pola **MVC**, dan konvensi commit `increment N` yang menyertainya

Slide ini membahas konsep. Langkah instalasi, setup proyek, dan latihan praktik lengkap ada di buku — bukan di slide.

Monolith

Monolith: Aplikasi yang seluruh lapisannya — antarmuka, logika bisnis, akses data — berjalan dalam satu basis kode dan satu proses deploy.

- Menambah fitur = menambah kode pada proyek yang sama
- Deploy pembaruan = mengganti satu unit dengan versi baru
- Sering disalahpahami sebagai "kode berantakan"
- Monolith terstruktur (mis. dengan MVC) tetap rapi
- Lawan katanya bukan "modular", melainkan **"terdistribusi"**

Microservices

Microservices: Arsitektur yang memecah aplikasi menjadi layanan independen, masing-masing berjalan dan di-deploy sendiri, saling berkomunikasi lewat jaringan.

- Setiap layanan: bahasa berbeda, jadwal deploy berbeda, skala sendiri-sendiri
- **Harganya:** 8 layanan = 8 proses deploy + 8 titik gagal + komunikasi jaringan antar-layanan
- Sepadan untuk **tim besar** dengan sistem berskala jutaan pengguna
- Untuk Simple POS: ongkos jauh melebihi manfaatnya

Serverless

Serverless: Kode dieksekusi sebagai fungsi-fungsi kecil yang hanya berjalan saat dipicu peristiwa, tanpa proses server yang menyala terus-menerus.

- Namanya menyesatkan: server tetap ada, hanya bukan tanggung jawabmu
- Ada jeda **cold start** saat fungsi lama tidak dipanggil
- Biaya dihitung per eksekusi, bukan per jam server menyala
- Cocok: beban kerja naik-turun tajam (mis. proses gambar saat upload)
- Kurang cocok: Simple POS yang butuh koneksi basis data konsisten

Perbandingan Ketiga Arsitektur

Arsitektur	Deployment	Kompleksitas Awal	Cocok Untuk
Monolith	Satu unit	Rendah	Simple POS, MVP, tim kecil
Microservices	Banyak unit independen	Tinggi	Sistem skala besar, tim besar
Serverless	Fungsi per event	Sedang	Beban kerja sporadis

Antarmuka (UI)

Logika Bisnis

Akses Data

Monolith

Layanan Produk

Layanan Pembayaran

Layanan Pengguna

Microservices

Mengapa Laravel?

MVC (Model-View-Controller): Model mengurus data, View mengurus tampilan, Controller mengurus alur permintaan di antara keduanya.

- Satu proyek PHP: routing, autentikasi, ORM, template — siap pakai
- Dibanding PHP polos: struktur MVC konsisten sejak baris kode pertama
- Dibanding framework microservices-first: tetap produktif untuk tim 1–2 orang
- Ekosistem paket kuat: **Sanctum** (API), **Excel** (impor/ekspor), **Cashier** (pembayaran)
- Filosofi mirip Django (Python) & Ruby on Rails: *convention over configuration*

Alur Satu Request Laravel



Setiap permintaan, apa pun alamatnya, selalu masuk lewat satu berkas yang sama: `public/index.php` .
Tidak ada berkas lain yang bisa diakses langsung lewat URL.

Menyiapkan Proyek: Kenapa SQLite?

- Basis data disimpan dalam **satu berkas biasa**
- Tanpa proses server terpisah yang harus dinyalakan & dikonfigurasi
- Migrasi bisa langsung dijalankan di menit pertama
- Simple POS berjalan di atas SQLite bahkan untuk demonstrasi kelas

Sebelum lanjut, pastikan empat alat berikut sudah terpasang: `php -v` (8.2+), `composer -V`, `node -v`, dan `git --version`.

Struktur Folder Laravel = Wujud MVC

Folder	Peran MVC	Isi
routes/web.php	Controller	Pendaftaran alamat URL
app/Http/Controllers/	Controller	Kelas pemroses request
database/migrations/	Model	Definisi skema basis data
resources/views/	View	Berkas Blade (mulai Bab 3)
vendor/	—	Paket Composer, tidak di-commit

Struktur ini bukan kebetulan — ia mewujudkan pola MVC yang sama dengan diagram alur request sebelumnya.

Konvensi `increment N`

- Repositori Simple POS tumbuh **commit demi commit**, masing-masing diberi label `increment N`
- Increment 1 = kerangka proyek kosong hasil `composer create-project`
- Urutan increment mengikuti urutan bab buku ini
- Membaca riwayat commit = membaca buku ini bab demi bab, dalam bentuk kode

Langkah `git init`, commit pertama, dan penjelasan lengkap konvensi ini ada di buku.

Rangkuman

- **Monolith** menyatukan seluruh lapisan dalam satu basis kode & satu deploy, cocok untuk skala Simple POS; **microservices** memecahnya dengan ongkos yang sepadan hanya untuk sistem besar; **serverless** cocok untuk beban kerja sporadis
- Laravel dipilih sebagai kerangka kerja Simple POS karena strukturnya konsisten sejak awal (MVC) dan produktif untuk tim kecil; **SQLite** dipilih sebagai basis data karena zero-setup — satu berkas, tanpa server terpisah
- Struktur folder Laravel mewujudkan pola **MVC**; konvensi commit `increment N` memetakan pertumbuhan aplikasi bab demi bab, selaras dengan urutan buku

Referensi & Diskusi

Dokumentasi resmi Laravel · Manual PHP

Kode lengkap: `github.com/dhanifudin/simple-pos`, branch `chapter-01`

Pertemuan berikutnya: HTTP & Arsitektur MVC